

Steer Engine: A Performance-Focused ECS Engine for Vehicular Action Games

Group Report for COMP 5531M

SICHENG CHEN*, SAMUIL GEORGIEV*, QIDAN GUO*, JUNXIAN LI*, CHENGHAO ZHU*, and RAFAEL KUFFNER DOS ANJOS[†], University of Leeds, United Kingdom

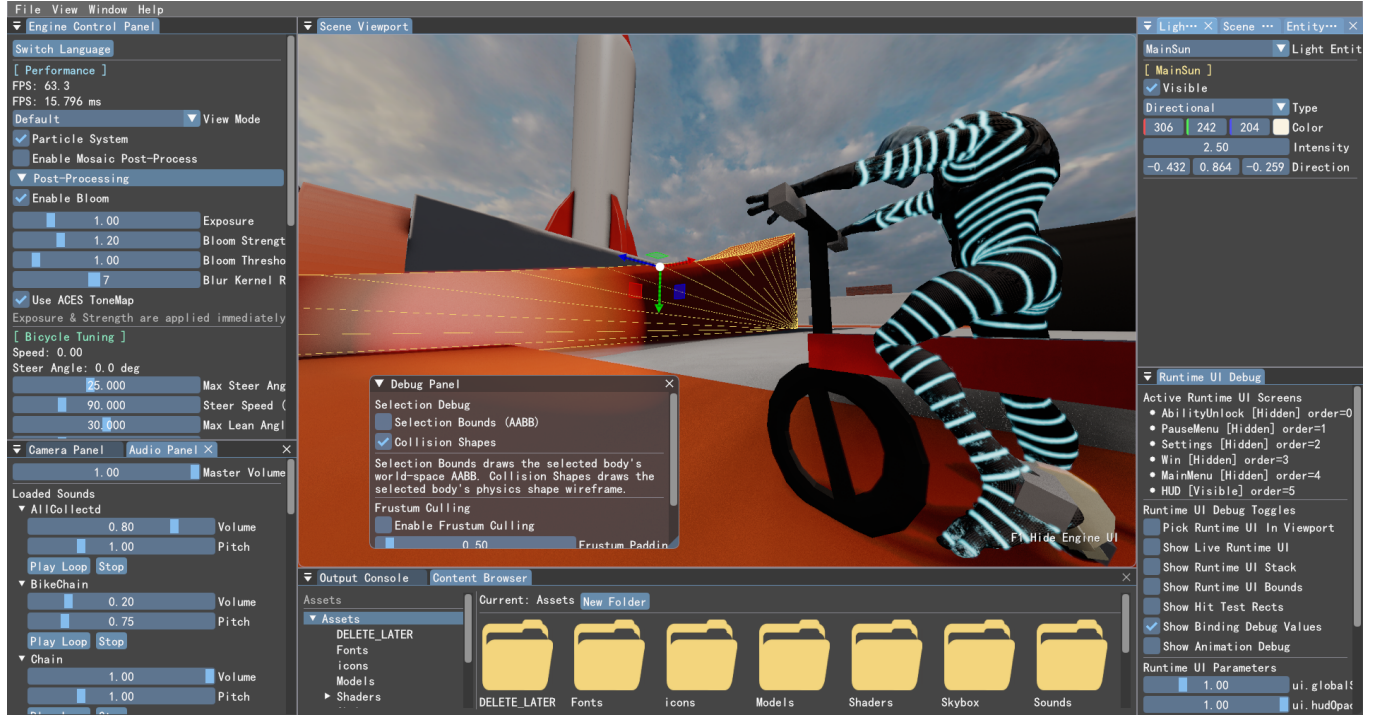


Fig. 1. Steer Engine: A performance-focused ECS engine showcasing Physically Based Rendering and Jolt Physics integration in a vehicular action game.

As game engines evolve into monolithic toolsets, they introduce unnecessary overhead for genre-specific applications. We present our custom, performance-focused game engine built entirely in C++, leveraging Vulkan for rendering, Flecs for Entity-Component-System (ECS) data locality, and Jolt Physics for deterministic vehicular simulation. By utilizing a pure ECS paradigm, we achieve high-performance entity management with negligible CPU bottlenecks. To demonstrate the engine, we developed a fast-paced bicycle action game requiring high-speed collision resolution and physics-driven mechanics. Our engine development focused on a robust core loop, hardware-accelerated Vulkan rendering, tuned rigid-body dynamics, and comprehensive tooling. The resulting Steer Engine reliably powers the vehicular demo, validating that targeted architectures can outperform generalized engines for specific domains.

*Sharing first authorship

[†]Supervisor

Authors' Contact Information: Sicheng Chen, sc22s3c@leeds.ac.uk; Samuil Georgiev, sc22sg@leeds.ac.uk; Qidan Guo, dhfc0067@leeds.ac.uk; Junxian Li, jmh0887@leeds.ac.uk; Chenghao Zhu, wqgc0081@leeds.ac.uk; Rafael Kuffner dos Anjos, r.kuffnerdosanj@leeds.ac.uk, University of Leeds, Leeds, West Yorkshire, United Kingdom.

CCS Concepts: • **Theory of computation** → Parameterized complexity and exact algorithms; • **Hardware** → Robustness; • **Computing methodologies** → Volumetric models.

Additional Key Words and Phrases: Game Engine, Vulkan, ECS, Flecs, Jolt Physics, Vehicular Simulation

1 Introduction

As game engines have evolved into monolithic toolsets (e.g., Unreal Engine, Unity), they inherently provide generalized architectures to cater to a massive variety of genres. While convenient, this generalization introduces unnecessary overhead, convoluted component hierarchies, and rigidity that can hinder performance-critical, genre-specific applications. We present our custom, performance-focused game engine built entirely in C++, leveraging the explicit graphics control of Vulkan, the rigorous data-locality of Flecs (an Entity-Component-System), and the deterministic physical simulation capabilities of Jolt Physics.

Our primary goal was to construct a scalable architecture specialized in vehicular mechanics and expansive environments, while

maintaining the flexibility to incorporate advanced stylisation and rendering features. By abandoning deep inheritance trees in favor of a pure ECS paradigm, we achieve high-performance entity management, capable of handling complex level assemblies with negligible CPU bottlenecks.

To demonstrate the engine’s capabilities, we developed a fast-paced bicycle action game. In this demo, the player controls a bicycle traversing a custom level geometry, aiming to collect scattered boosters. Once all boosters are collected, the player acquires enhanced speed and jumps off a designated ramp to clear the level’s enclosing walls. This action effectively completes the stage and seamlessly transitions the player to the next.

Our engine development focuses on four main pillars:

- (1) **Core Architecture:** An optimized, ECS-driven game loop handling engine life-cycle, event dispatching, and scene management.
- (2) **Physically Based Rendering (PBR):** A Vulkan-based backend providing hardware-accelerated Shadow Mapping, Alpha Masking, Post-Processing, and debug visualisations.
- (3) **Rigid Body Dynamics:** Integration of Jolt Physics with a specialized Vehicle Constraint system for robust, tunable bicycle control.
- (4) **Tooling & Scalability:** ImGui integration, input action mapping, and level assembly pipelines.

While early prototypes lacked full audiovisual polish, the core engine loop now integrates a comprehensive Audio System powered by miniaudio, a Particle System for visual effects, and a Skeletal Animation System featuring Inverse Kinematics. This robust foundation reliably powers a fully playable, physics-driven vehicular demo, demonstrating scalable performance.

2 Background Research

2.1 Game Engines and Architectural Patterns: OOP vs. ECS

Modern game development largely depends on foundational software frameworks known as game engines. Historically, engines have relied heavily on deep inheritance structures and Object-Oriented Programming (OOP) to manage entities, a pattern extensively documented by Jason Gregory in his seminal work *Game Engine Architecture* [Gregory 2018]. For example, a game entity might inherit from `GameObject`, which inherits from `Transformable`, which inherits from `BaseNode`.

Drawbacks of OOP Engines: While OOP provides intuitive hierarchical modeling, deep inheritance often leads to the "Deadly Diamond of Death" and rigid class structures that are difficult to refactor. More critically, OOP engines suffer from poor memory data-locality. Objects are scattered across the heap, causing frequent CPU cache misses when iterating over large numbers of entities, which severely degrades performance in environments crowded with interactive elements. This limitation is heavily criticized in Richard Fabian’s *Data-Oriented Design* [Fabian 2018], which advocates for memory contiguity.

Advantages of ECS (Flecs): To overcome these limitations, our engine embraces the Entity-Component-System (ECS) pattern,

specifically utilizing Sander Mertens’ *Flecs* [Mertens 2025], similar to the robust ECS implementation seen in Blizzard’s *Overwatch* gameplay architecture [Ford 2017]. In ECS, entities are merely lightweight integer IDs, components are plain-old-data (POD) structs, and systems contain the logic that iterates over specific component combinations. The primary importance of ECS is its excellent CPU cache locality. Components of the same type are stored contiguously in memory arrays, enabling highly efficient batch processing. This architecture fundamentally justified our choice of Flecs, allowing us to manage massive amounts of dynamic entities (e.g., thousands of custom level particles and instanced geometries) without traditional CPU bottlenecks.

2.2 Real-Time Graphics: Vulkan vs. OpenGL/DirectX

Real-time rendering APIs serve as the bridge between engine logic and the GPU. Legacy APIs like OpenGL and earlier versions of DirectX (DX11) manage a significant amount of state implicitly behind the scenes.

Drawbacks of Legacy APIs (OpenGL/DX11): While easier to learn, OpenGL suffers from unpredictable driver overhead and opaque thread management. The driver implicitly handles memory allocation and synchronization, taking away optimization opportunities from the developer. This often results in CPU-bound rendering bottlenecks, a well-known limitation of OpenGL as described in the *OpenGL SuperBible* [Sellers 2015].

Advantages of Explicit APIs (Vulkan): The industry has shifted toward explicit APIs like Vulkan and DirectX 12. Vulkan gives developers granular control over GPU memory, explicit synchronization barriers, and multi-threaded command buffer dispatching, a paradigm shift popularized by high-performance engines like idTech 6 in *DOOM Eternal* [Gneiting 2016; id Software 2020]. Despite the steep learning curve and highly verbose setup required, Vulkan was chosen for our engine because it completely eliminates unpredictable driver overhead. It allows our custom `RenderSystem` to meticulously define sub-passes, handle explicit image layout transitions, and construct precise rendering pipelines. These capabilities are essential for implementing performant Physically Based Rendering (PBR), hardware Percentage-Closer Filtering (PCF), and dynamic shadow mapping without sacrificing frame rates.

2.3 Vehicular Physics: Jolt vs. PhysX and Bullet

Simulating vehicular dynamics in real-time requires a robust physics solver capable of handling suspension, wheel friction, and high-speed collision detection without erratic jittering.

Drawbacks of Generalised Physics Engines (PhysX/Bullet): NVIDIA’s PhysX and the Bullet physics engine are the traditional industry standards. PhysX provides extensive features including soft bodies, fluid dynamics, and cloth simulation, as detailed in the NVIDIA PhysX SDK [NVIDIA 2025] and seen in titles like *Cyberpunk 2077* [CD Projekt Red 2020]. However, this generalization introduces significant overhead. Bullet, while open-source and modular, can sometimes struggle with multithreaded scalability and deterministic rigid-body resting states under heavy loads, an issue

sometimes encountered with the open-source Bullet Physics library [Coumans 2025].

Advantages of Jolt Physics: Jolt Physics [Rouwe 2025] has recently emerged as a high-performance alternative explicitly designed to prioritize speed, deterministic rigid-body simulation, and multithreaded scalability, as engineered by Jorrit Rouwe for *Horizon Forbidden West* [Guerrilla Games 2022; Rouwe 2021]. Jolt deliberately omits soft-body and fluid simulations to maintain a lightweight core. For our project, the deciding factor was Jolt’s built-in VehicleConstraint system. It provides a highly configurable interface for raycast-based wheel suspension, damping, and custom friction simulation, heavily inspired by arcade-racer mechanics seen in commercial titles like *Need for Speed Unbound* [Criterion Games 2022; Mackenzie 2022] and *Mario Kart 8* [Nintendo EAD 2014]. This explicitly tailored rigid-body focus made Jolt the ideal candidate to handle the high-speed ramps, collision shapes, and dynamic torque requirements of our bicycle mechanics without the bloat of an overarching generalised physics system.

3 Methodology: Engine Architecture & Implementation

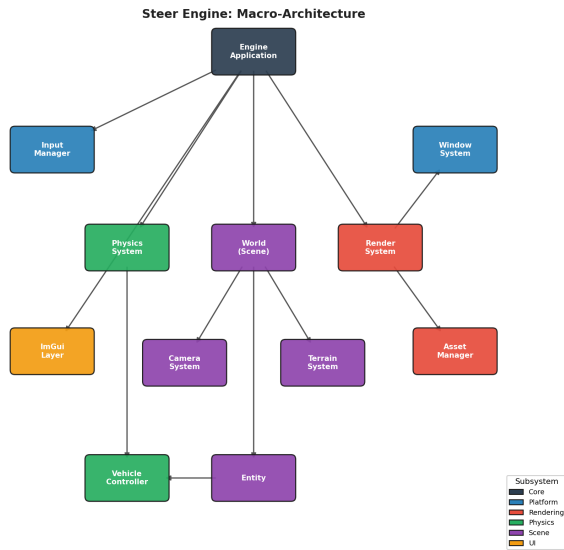


Fig. 2. **Simplified Engine Macro-Architecture.** Directed edges represent ownership or primary data-flow relationships. The EngineApplication orchestrates five top-level subsystems. The World manages scene-graph entities, camera, and terrain, while the RenderSystem owns the Vulkan backend and asset pipeline.

The engine’s macro-architecture, illustrated in Figure 2, is designed around a decoupled, data-driven methodology that isolates the main loop, scene management, physics simulation, and rendering pipelines into distinct subsystems. The core structural interactions can be summarised as follows:

- **EngineApplication:** Acts as the overarching controller. It owns and coordinates the primary subsystems: InputManager, PhysicsSystem, RenderSystem, the ImGuiLayer, and the active World.

- **World:** Functions as the scene manager, holding the CameraSystem, TerrainSystem, and a collection of Entity objects. It bridges gameplay logic and rendering by processing entities and exposing them via GetRenderables().
- **Entity:** Represents an individual game object in the world. Instead of complex inheritance, entities act as containers for components like transforms, mesh references, and physics bodies.
- **RenderSystem:** Drives the Vulkan backend. It references the WindowSystem for surface management, retrieves geometry and textures via the AssetManager, and compiles frame commands for GPU submission.
- **PhysicsSystem:** Encapsulates the Jolt Physics engine. It owns the VehicleController and is strictly responsible for updating spatial data and handling collision constraints for entities that possess a PhysicsBody.

3.1 Core System and ECS Integration (Flecs)

The engine’s lifecycle is governed by the Application class, which acts as the central orchestrator. It initializes the core subsystems: Windowing via GLFW [Geelnard 2025], Renderer, Input, Events, Physics, and the SceneManager.

The core logic revolves around **Flecs**, our chosen ECS backend.

Justification for Flecs: In a high-speed vehicular game like our bicycle demo, the engine must constantly update the transforms of the player, environmental triggers (boosters), and potentially hundreds of dynamic particles without hitching. Flecs provides rigorous data locality by packing components into memory arrays based on entity archetypes. This allows systems to iterate through homogeneous blocks of memory, maximizing CPU cache hits. This decoupled architecture inherently separates data from logic, ensuring that gameplay code remains cleanly separated from rendering and physics integration.

3.1.1 Core Component Structs. To fully exploit Flecs’ cache locality, our components are defined as tightly packed, plain-old-data (POD) C++ structs without virtual functions. This ensures they can be copied directly in contiguous memory. Some of our core components include:

```

struct LocalTransform { glm::mat4 matrix; };
struct WorldTransform { glm::mat4 matrix; };
struct MeshComponent { uint32_t meshIndex; };
struct PhysicsBody { uint32_t bodyID; };
struct EntityState {
    bool should_render;
    bool has_physics;
};

```

Each component is a plain-old-data (POD) struct containing only scalar or fixed-size matrix fields. They use no virtual functions and require no heap allocations. For example, LocalTransform and WorldTransform each wrap a single 4×4 affine matrix, while MeshComponent and MaterialComponent store lightweight 32-bit indices into the engine’s unified geometry and material registries. The PhysicsBody component holds a Jolt BodyID handle alongside a boolean flag indicating dynamic status, and the RenderBatch

component records vertex offsets, index offsets, and index counts into the shared Vulkan device-local buffer.

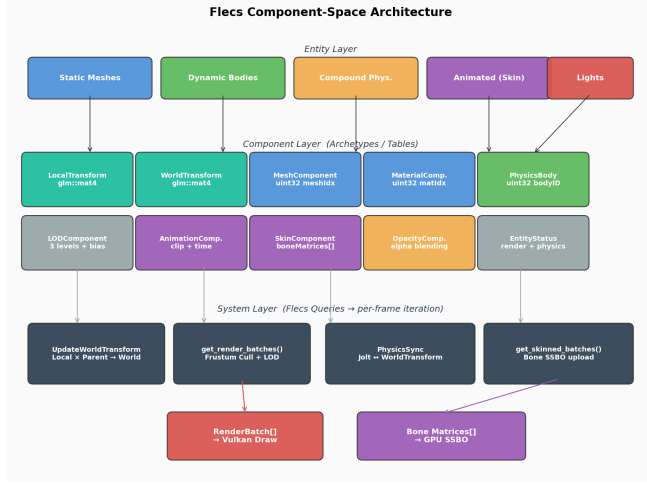


Fig. 3. **Flecs Component-Space Architecture.** Entities are grouped into archetypes based on their component signature. Components of identical type are packed contiguously, maximising CPU cache utilisation during system iteration. Arrows indicate the data flow from the SceneManager parse stage through archetype assignment to per-frame system queries.

These lightweight structs form the archetypes for our entities. For example, a static renderable obstacle consists of a `WorldTransform`, a `MeshComponent`, a `MaterialComponent`, and a `RenderBatch`. If it collides, it also contains a `PhysicsBody`.

3.1.2 System Queries and Scheduling. The power of Flecs lies in its ability to query entities that possess specific intersections of components. We define system queries to explicitly request access to arrays of components, which the ECS executes as batched arrays.

For instance, the `UpdateWorldTransform` system requests mutable access to `WorldTransform` and read-only access to `LocalTransform`. For each matched entity, the system computes the world-space matrix W_e by composing the parent’s cached world matrix with the entity’s local matrix L_e . Root entities simply use their local matrix directly. Because Flecs iterates over contiguous archetype tables, this hierarchical propagation executes with minimal cache misses even for deeply nested scene graphs.

To guarantee deterministic engine behaviour and eliminate frame-tearing or physics-jitter, we explicitly defined the execution phases of our ECS pipeline. System scheduling was rigorously managed to ensure data mutations occur in a strictly linear pipeline:

- (1) **Input Phase:** Parses hardware states and translates them into action events.
- (2) **Pre-Physics Phase:** Applies gameplay logic (e.g., booster trigger deletions) and input torque to the bikeController.
- (3) **Physics Tick:** Steps the Jolt simulation forward based on the fixed time-step.
- (4) **Post-Physics Transform Sync:** A critical system that reads the simulated `JPH::BodyID` transforms and overwrites the

`WorldTransform` components of dynamic entities. This explicitly occurs *before* rendering.

- (5) **Render Tick:** The `RenderSystem` iterates over the now accurately positioned `WorldTransform` and `RenderBatch` entities to populate the Vulkan command buffers.

3.1.3 Level Assembly and Parsing Pipeline. The `SceneManager` is responsible for translating human-readable level layouts into optimized Flecs archetypes. Our level assembly pipeline is designed to parse incoming scene data configurations (such as simplified GLTF structures or custom JSON maps).

When parsing a level file, the `SceneManager` iterates over the defined objects and immediately instantiates them as lightweight Flecs IDs. It then dynamically attaches components:

- If the parsed node contains vertex data, the geometry is uploaded to the unified Vulkan staging buffer, and a `RenderBatch` and `MeshComponent` are assigned.
- If the node is flagged as a solid collider (like the jump ramp or the enclosing walls), the `SceneManager` automatically generates a static triangle mesh collider utilizing Jolt’s shape generator and attaches the resulting `PhysicsBody` component to the entity.
- If the node is flagged as a booster, it receives a trigger volume component and an event-listener callback.

This data-driven pipeline guarantees that our levels can be authored in external 3D software and instantly digested into the tightly-packed, cache-friendly architecture the rest of the engine relies upon.

3.2 Rendering System

The Vulkan rendering pipeline forms the visual backbone of the engine. Based on the requirements from early development tasks, we transitioned from basic rasterisation to a modular, multi-pass architecture managed by the `RenderSystem`.

3.2.1 Vulkan Dynamic Rendering Setup. To modernize the engine, our Vulkan backend utilizes the `VK_KHR_dynamic_rendering` extension from the Vulkan 1.3 Specification [Khronos 2025] to explicitly control state without the legacy rigid structures of `VkRenderPass` and `VkFramebuffer` objects. This significantly simplified our code-base when managing complex multi-pass rendering.

Instead of pre-baking framebuffers, we begin rendering by populating `VkRenderingAttachmentInfo` structures per frame. This structure specifies the `imageView`, the `imageLayout` (e.g., `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`), and the `loadOp/storeOp` for each attachment. We then bind these attachments to a `VkRenderingInfo` struct, which defines the render area and layer count.

At the start of each frame, the renderer populates per-attachment descriptors specifying the target image view, the desired image layout (e.g., `COLOR_ATTACHMENT_OPTIMAL`), and the load/store operations. These descriptors are aggregated into a single rendering info structure that defines the render area, layer count, colour attachment array, and depth attachment reference. The command buffer then begins a dynamic rendering scope against these descriptors, eliminating the need for statically pre-baked framebuffer objects.

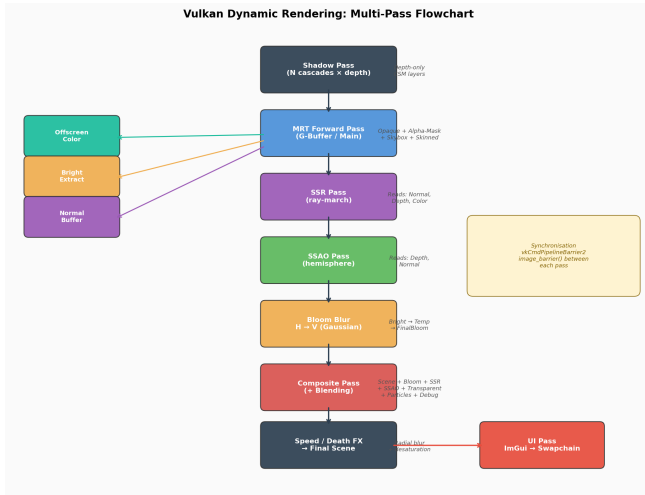


Fig. 4. **Render Pass Flowchart.** Data flow through the engine’s multi-pass Vulkan pipeline. Each pass operates on explicit attachment descriptors. Pipeline barriers enforce layout transitions and execution dependencies between the Shadow, Forward, Alpha-Mask, and Post-Processing stages.

We rely heavily on explicit synchronization primitives. Before calling `vkCmdBeginRendering`, we insert pipeline barriers using `vkCmdPipelineBarrier` to explicitly transition image layouts (e.g., from `VK_IMAGE_LAYOUT_UNDEFINED` to optimal attachment layouts) ensuring execution dependencies across our Shadow, Main Forward, and Post-Processing passes are strictly respected without GPU stalls.

3.2.2 Data Management: UBOs and Descriptor Sets. Supplying data to the shaders efficiently is critical for performance. We heavily utilize Uniform Buffer Objects (UBOs) to pass global scene parameters (View Matrix, Projection Matrix, Light Position, and Camera Position) to the vertex and fragment pipelines.

std140 Memory Alignment: A significant C++ implementation challenge was guaranteeing memory alignment between the CPU data structs (using GLM) and the Vulkan GLSL shader expectations. We strictly adhered to the std140 layout rules. For instance, in our C++ struct, a `glm::vec3` (12 bytes) must be padded to 16 bytes. We explicitly added `alignas(16)` to our struct members to prevent subtle data corruption where Vulkan reads offset bytes:

```
struct alignas(16) ObjectPC {
    glm::mat4 transform;           // Offset 0, Size 64
    glm::vec4 baseColorFactor;     // Offset 64, Size 16
    glm::vec4 emissiveFactor;      // Offset 80, Size 16
    float metallicFactor;          // Offset 96, Size 4
    float roughnessFactor;         // Offset 100, Size 4
};
```

Each field in the global uniform buffer is aligned to a 16-byte boundary in accordance with the `std140` layout specification. In particular, three-component vectors (12 bytes) are padded to 16 bytes, preventing byte-offset mismatches between the host-side C++ struct and the device-side GLSL interface block. The buffer supplies the camera’s View matrix $\mathbf{V}$, the Projection matrix $\mathbf{P}$, the directional

light position $\mathbf{p}_{\text{light}}$, and the camera position $\mathbf{p}_{\text{cam}}$ to every shader stage via a single descriptor binding.

Descriptor Pools and Sets: To map these UBOs and our PBR material textures to the shaders, we manage an engine-wide `VkDescriptorPool`. The pool is pre-allocated to support thousands of sets (e.g., combining `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` and `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`). When the `SceneManager` instantiates a new entity with a `MaterialComponent`, the `RenderSystem` allocates a `VkDescriptorSet` from the pool, writes the specific Albedo, Normal, Roughness, and Metalness `VkImageViews` into it via `vkUpdateDescriptorSets`, and caches it. During the render loop, `vkCmdBindDescriptorSets` is called to bind the appropriate material parameters immediately before the draw call.

3.2.3 Render Pass Layout. The pipeline is logically separated into distinct stages, each executed linearly with strict pipeline barriers ensuring safe memory transitions between color and depth attachments.

Pass	Type	Function / Attachments
Shadow Pass	Depth-only	Cascaded light-space depth generation
Forward MRT	G-Buffer	Opaque, Alpha-Mask, Skybox; Color + Bright + Normal
SSR Pass	Post-Process	Screen Space Reflections based on G-Buffer
SSAO Pass	Post-Process	Ambient Occlusion via depth/normal sampling
Gaussian Blur	Compute-like	Horizontal/Vertical blur on Brightness attachment
Composite	Main Blending	Scene + Bloom + SSR + SSAO + Transparency + Particles
Post-FX	Final Resolve	Speed-blur, Death-desaturation, Bloom Resolve
UI Pass	Overlay	Dear ImGui render submission to Swapchain

Table 1. Modular Multi-Pass Rendering Pipeline Layout.

- (1) **Shadow Pass (Cascaded Orthographic):** An off-screen depth-only pass capturing the scene from the directional light's perspective. To ensure maximum shadow resolution near the player without wasting texels on distant geometry, we dynamically calculate a Light Space Matrix. We achieve this by extracting the eight corners of the camera's view frustum in world space. We then transform these corners into the light's view space using a LookAt matrix (positioned at the light's origin, looking at the frustum center). We calculate the minimum and maximum X, Y, and Z coordinates of these transformed corners to define a tight bounding box. These bounds directly populate an orthographic projection matrix (`glm::ortho`). Multiplying the Light Projection by the Light View matrix yields the final Light Space Matrix sent to the vertex shader via our `GlobalUBO`.

For each cascade segment i , we construct a per-segment perspective matrix spanning $[z_{i-1}, z_i]$ and invert the combined view-projection to un-project the eight canonical NDC corners into world space via perspective division. Their centroid defines the look-at target for the light view matrix $\mathbf{V}_{\text{light}}$, while their axis-aligned bounding box in light space yields a tight orthographic projection $\mathbf{P}_{\text{ortho}}$. The final per-cascade light-space matrix is simply $\mathbf{T}_{\text{light}}^{(i)} = \mathbf{P}_{\text{ortho}} \cdot \mathbf{V}_{\text{light}}$, which is uploaded to the vertex shader via the global UBO.

- (2) **Main Forward Pass:** Iterates over all opaque RenderBatch entities. It utilizes the standard Camera View-Projection matrices. The fragment shader receives the interpolated vertex positions in light-space, applies perspective division, and samples the generated Shadow Map (via `sampler2DShadow` and `textureProj`) to determine the shadow occlusion factor before computing the PBR lighting.
- (3) **Alpha-Masked Pass:** Specifically targets geometry requiring transparency (like chain-link fencing and foliage). Traditional alpha-blending requires strict back-to-front depth sorting on the CPU, which breaks Flecs' cache-friendly iteration order. Instead, we use alpha-masking: the fragment shader samples the albedo texture's alpha channel and invokes the discard intrinsic whenever $\alpha_{\text{albedo}} < 0.5$, terminating execution and preventing depth-buffer writes. Fragments passing the threshold are evaluated through the full Cook-Torrance BRDF. This strategy avoids CPU-side back-to-front sorting, preserves Flecs' cache-friendly iteration order, and still benefits from hardware early-depth rejection on opaque regions.
- (4) **Post-Processing & Debug Pass:** Renders a single full-screen triangle bypassing the vertex buffer. It reads the resolved Swapchain Image and overlays any requested debug visualizations (Overdraw heatmaps, wireframes) before presentation.

3.2.4 Physically Based Rendering (PBR) & Cook-Torrance BRDF. Our forward shading pass implements a physically inspired model based on the Cook-Torrance microfacet BRDF [Cook and Torrance 1982]. The outgoing light L_o is calculated by integrating the ambient light, emissive colour, and directional light contributions. For a single directional light, the core reflection equation is:

$$L_o = L_e + L_{\text{ambient}} + f_r(\mathbf{l}, \mathbf{v}) c_{\text{light}} (\mathbf{n} \cdot \mathbf{l})^+$$

where $\mathbf{n}$ is the surface normal, $\mathbf{l}$ the light direction, and $\mathbf{v}$ the view direction. The BRDF f_r combines a Lambertian diffuse lobe with a microfacet specular lobe:

$$f_r(\mathbf{l}, \mathbf{v}) = \frac{c_{\text{mat}}}{\pi} (1 - F)(1 - M) + \frac{D F G}{4 (\mathbf{n} \cdot \mathbf{v})^+ (\mathbf{n} \cdot \mathbf{l})^+}$$

The three specular terms are standard choices from the literature. The **Normal Distribution Function** D uses the Beckmann distribution [Beckmann and Spizzichino 1963], parameterised by $\alpha = \text{roughness}^2$, which models the statistical orientation of surface microfacets. The **Fresnel factor** F is evaluated via Schlick's approximation [Schlick 1994], $F = F_0 + (1 - F_0)(1 - \mathbf{h} \cdot \mathbf{v})^5$, where the base reflectivity F_0 interpolates between a dielectric constant (0.04)

and the material albedo based on the metalness M . The **Geometry term** G uses the Cook-Torrance masking-shadowing function, which attenuates the specular contribution when microfacets occlude each other. The diffuse lobe is energy-conserving: it is scaled by $(1 - F)(1 - M)$, cancelling the Lambertian term entirely for fully metallic surfaces.

3.2.5 Shadow Mapping Improvements. Following earlier feedback regarding shadow map aliasing and incorrect alpha geometry rendering, the shadow pipeline was entirely refactored. We now explicitly manage two distinct shadow map shaders: one for standard opaque geometry and another explicitly evaluating alpha thresholds for masked textures (e.g., foliage). Hardware Percentage-Closer Filtering (PCF) is properly utilized via `sampler2DShadow` binding and `textureProj` lookups, resolving previous artifacts and viewport/scissor matrix misalignments introduced by naive 3x3 software filtering.

3.3 Physics System

The `PhysicsSystem` encapsulates the Jolt Physics engine. We synchronize the engine's logical ECS `WorldTransform` with Jolt's `BodyInterface` per physics tick.

Justification for Jolt Physics: Simulating a fast-moving bicycle traversing ramps and collecting triggers demands high-frequency, deterministic updates. Traditional physics engines like PhysX are feature-rich but introduce massive overhead for unused modules (e.g., cloth or fluids). Jolt Physics is laser-focused on rigid-body determinism and multithreading. Most crucially, Jolt's native `VehicleConstraint` system provides a highly tunable raycast suspension model out-of-the-box. This explicitly tailored rigid-body focus made Jolt the ideal candidate to handle the high-speed ramps and collision shapes of our mechanics.

3.3.1 Broad-phase and Narrow-phase Collision. To optimize collision detection across our expansive level geometry, Jolt is configured with a two-tiered spatial partitioning system.

- **Broad-phase:** We utilize Jolt's Bounding Volume Hierarchy (BVH) to rapidly cull pairs of objects that are too far apart to intersect. We categorize our objects into distinct Broad-phase layers: `NON_MOVING` (walls, floors) and `MOVING` (the player, dynamic particles). This ensures static geometry never needlessly tests against other static geometry.
- **Narrow-phase:** For objects passing the Broad-phase bounding box intersection, the Narrow-phase executes precise geometric overlap tests. For our static level, the `SceneManager` automatically parses imported mesh geometry into optimized Jolt `MeshShape` objects. The player utilizes convex hulls.

3.3.2 Bridging Jolt and Flecs. A major architectural challenge was mapping Jolt's deterministic physical bodies back to our ECS entities so game logic could respond to collisions. We achieved this by utilizing Jolt's `Body::SetUserData` function. When the `PhysicsSystem` creates a rigid body, we explicitly cast the Flecs Entity ID (a 64-bit integer) and store it within the Jolt body's user data payload: When a rigid body is created, the 64-bit Flecs entity identifier is embedded

directly into the Jolt body's user-data field. This establishes a bijective mapping $f : \text{BodyID} \rightarrow \text{EntityID}$ that can be resolved in $O(1)$ during collision callbacks.

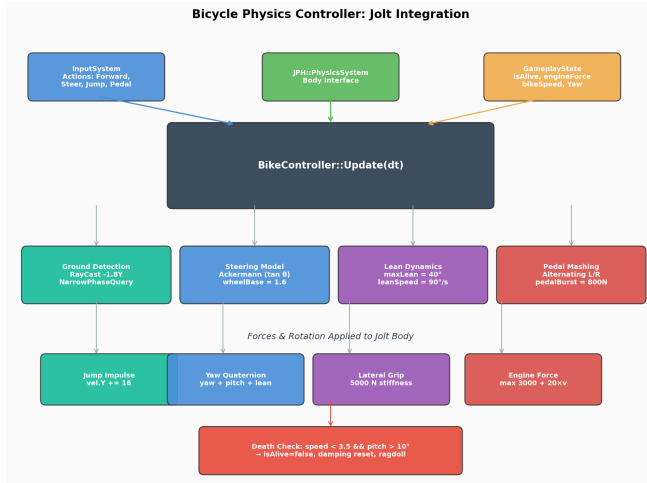


Fig. 5. **Vehicle Constraint Node Setup.** The Jolt VehicleConstraint hierarchy showing the chassis body, the two raycast wheel anchors, suspension spring parameters, and the user-data link back to the Flecs entity graph. Arrows indicate the per-tick data flow: input torques flow in, simulated transforms flow out.

Consequently, when Jolt's collision listener fires a contact callback, we retrieve this payload, cast it back to a Flecs entity, and dispatch an engine-wide CollisionEvent. This elegantly bridges the object-oriented Jolt solver with our data-oriented Flecs architecture.

3.4 Supporting Systems

3.4.1 Input System and Action Mapping. To support a polished user experience, our InputSystem abstracts hardware-specific APIs into semantic gameplay actions. Rather than hardcoding if (glfwGetKey(GLFW_KEY_W)), systems poll an ActionMap for conceptual triggers like "Accelerate" or "Steer."

The InputSystem hooks directly into GLFW's callback functions. For keyboard and mouse inputs, raw GLFW key codes are translated and stored in an internal state array, tracking whether a key is "Pressed," "Held," or "Released" during the current frame. This state is then evaluated against a user-defined configuration file that maps physical inputs to the abstract Action names.

A significant challenge arose when implementing controller support, specifically mapping input axes within our Linux development environment via the glfwGetJoystickAxes API. This ensures precise control over the bikeController's steering constraints.

3.4.2 Event System and the Observer Pattern. Inter-system communication within a monolithic engine often leads to dangerous circular dependencies. To guarantee decoupled architecture, we implemented a robust Event System utilizing a C++ implementation of the Observer pattern.

Immediate dispatch could disrupt the memory state if an event triggers an entity deletion mid-iteration. To prevent this, we utilize a thread-safe **Event Queue** instead. When the PhysicsSystem detects a booster collision, it does not immediately call a function in the SceneManager. Instead, it allocates a CollisionEvent object and pushes it onto the back of the queue.

The EventSystem maintains a type-indexed subscriber map. Systems register callback functions keyed by EventType during engine initialisation. Rather than dispatching events immediately, events are enqueued into a thread-safe FIFO buffer protected by a mutex. This prevents the iterator state from being corrupted if a handler mutates the Flecs world mid-query. During the dedicated "Event Processing" phase of the main loop, the dispatcher drains the queue and broadcasts each event to all registered subscribers in registration order. If a subscriber marks the event as "handled", propagation halts, implementing a priority-based consumption model.

Systems register themselves as listeners during the Application initialization phase by binding function pointers (using std::bind or lambda expressions) to specific event types via a central EventDispatcher. During the designated "Event Processing" phase of our main game loop, the dispatcher iterates over the queue, broadcasting each event to all registered listeners simultaneously. This deferred execution ensures that circular dependencies are impossible: the physics system does not need to #include "SceneManager.hpp", it merely broadcasts that an event occurred into the void, allowing the Scene Manager to listen and react entirely independently.

3.4.3 UI System and Vulkan Injection. A modern engine requires robust, real-time developer tooling. We integrated the widely-adopted Dear ImGui library [Cornut 2025] created by Omar Cornut to handle all User Interface requirements.

Because our Vulkan rendering pipeline is strictly controlled via dynamic rendering passes, injecting the UI required meticulous orchestration. The UISystem operates as the absolute final sub-pass of the RenderTick. After the Main Forward pass and the Alpha-Masked pass have completed, we initiate a final VkRenderingInfo pass specifically targeted at the swapchain image. We pass the active Vulkan command buffer directly to ImGui_ImplVulkan_RenderDrawData, overlaying the UI graphics on top of the 3D scene immediately before the vkQueueSubmit call.

This immediate-mode GUI proved invaluable for debugging. Our custom debug windows tap directly into the engine's core metrics, exposing:

- **Frametimes:** Tracking CPU logic ticks versus GPU render latency in milliseconds.
- **Draw Calls:** Displaying the exact number of vkCmdDrawIndexed submissions per frame, allowing us to actively verify that Frustum Culling is working correctly.
- **Physics Tuning:** Real-time sliders linked directly to the Jolt VehicleConstraint parameters, enabling us to tweak suspension stiffness and friction curves without recompiling the C++ codebase.

3.4.4 Audio System (Miniaudio). To provide immersive auditory feedback without overwhelming the CPU, we implemented a dedicated AudioSystem leveraging the miniaudio single-header library. Miniaudio was selected for its minimal footprint and lack of external dependencies, directly aligning with our performance-focused ethos. The system provides an abstraction layer capable of parsing generic sound files and managing spatial/one-shot playback. Sound states (such as base volume, pitch, and runtime volume multipliers) are managed per-sound within a global hash map, ensuring rapid localized lookups during runtime events (such as booster collection or engine revving).

3.4.5 Skeletal Animation and Inverse Kinematics (IK). While early development utilized static meshes, our engine now supports full Skeletal Animation evaluation. The AnimationSystem reads keyframe samplers and translates bone transformations. To dynamically integrate the player character with the vehicular simulation, we implemented a 2-bone Inverse Kinematics (IK) solver. The RiderIKComponent is attached to the character entity, defining specific chains (e.g., shoulder-elbow-hand). During the AnimationSystem update tick, the solver actively computes joint rotations to ensure the rider’s hands remain firmly attached to the bicycle’s handlebars, dynamically adapting to the bicycle’s LocalTransform and steering vectors.

3.4.6 Particle System Integration. Visual feedback is essential for game feel, particularly during high-speed actions. We implemented a custom ParticleSystem supporting GPU-instanced rendering to minimize draw calls. Particles are emitted via configurable shapes (Cone, Sphere, Box) using randomized seed values to calculate unique lifetime, scale, and velocity vectors per particle. Memory allocations for particle pools are explicitly managed through Vulkan bindings via volk, allowing the engine to spawn thousands of dynamic particles (e.g., booster explosions or tire smoke) with negligible performance degradation.

Each particle is initialised with randomised lifetime, scale, and speed values drawn uniformly from artist-configurable [min, max] ranges using independent $\mathcal{U}(0, 1)$ samples. The initial velocity direction depends on the emitter shape: cone emitters perturb the up-axis by a spread angle σ , while sphere and box emitters apply analogous shape-specific sampling strategies. This randomisation ensures visually varied, non-repetitive particle clouds from a single emitter definition.

3.4.7 Asset Upload Pipeline. Modern Vulkan rendering strictly forbids the GPU from directly reading vertex and index data from system RAM if performance is a priority. To achieve optimal draw call performance, our engine implements a dedicated Asset Upload Pipeline utilizing Vulkan Staging Buffers.

When the SceneManager parses a level, the raw vertex and index data is initially loaded into system memory. We first allocate a temporary staging buffer using the `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` | `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` flags. This allows the CPU to use `vmaMapMemory` to directly `memcpy` the raw geometry data into the buffer.

However, this memory is slow for the GPU to read during rendering. Therefore, we allocate the final destination buffer on the GPU explicitly using the `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` flag, which represents the fastest VRAM available. We then record a one-time-submit command buffer to execute a `vkCmdCopyBuffer` command:

The upload follows a three-stage process: (1) allocate a host-visible staging buffer flagged for sequential CPU writes, (2) map, copy, and unmap the raw data into the staging buffer, and (3) record a one-time-submit command buffer containing a buffer-to-image (or buffer-to-buffer) copy targeting a device-local allocation. A fence synchronizes CPU-side cleanup: once the GPU signals transfer completion, the staging buffer is immediately destroyed, leaving geometry and texture data residing exclusively in the fastest available VRAM.

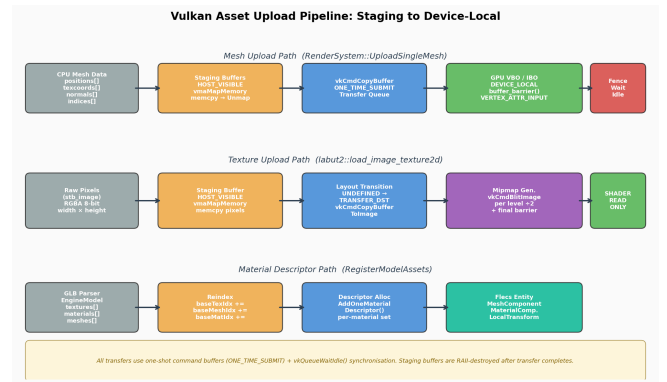


Fig. 6. **Vulkan Asset Upload Pipeline.** Data flows from the SceneManager parser through a host-visible staging buffer, across a one-shot transfer command, and into device-local GPU memory. The fence-guarded handoff ensures the staging allocation is released immediately after the transfer completes.

After the transfer command is submitted and the GPU signals a fence upon completion, the CPU safely destroys the temporary staging buffer. This ensures that during the Main Forward Pass, the vertex shaders are pulling geometry exclusively from ultra-fast, dedicated GPU memory, entirely eliminating PCIe bus bottlenecks during the render loop.

3.5 Implementation: The Biker Demo

The bicycle action game serves as the primary technical demonstration of our engine’s capabilities. The game loop emphasizes the synergy between the ECS logic, the vehicle physics, and the trigger mechanisms.

3.5.1 Level Assembly Pipeline & Geometry. The level is assembled via the SceneManager, which orchestrates the loading of static objects (the ground, enclosing walls, and the ramp) and dynamic entities (the bicycle, the player, and collectible boosters). The level assembly pipeline parses custom scene formats and dynamically spawns Fecs entities, assigning rendering and collision components on the fly.

The environment uses alpha-masked textures for fencing to demonstrate the multi-pass shadow pipeline, ensuring the lighting remains grounded and realistic even with complex overlapping geometry.

3.5.2 Game Logic and the Biker Controller. The player operates a physically simulated bicycle using input actions bound to acceleration and steering. To make the bicycle feel responsive, the `bikeController` dynamically manipulates the `VehicleConstraint` based on user input.

Torque Application and Steering Vectors: When the player presses the “Accelerate” input, we do not simply set a linear velocity. Instead, we apply a rotational torque to the rear wheel of the `VehicleConstraint`, calculated along the chassis forward vector $\hat{d} = (-\sin \psi, 0, -\cos \psi)$ derived from the current yaw ψ . The engine force F_{eng} follows a burst-decay model: each pedal input increases the force by a burst impulse F_{burst} scaled by a slope penalty κ_{slope} , clamped to F_{max} . Upon release, the force decays linearly at rate r_{decay} . To prevent unrealistic lateral sliding, a corrective grip force $F_{\text{lateral}} = -v_{\text{lat}} \cdot k_{\text{grip}} \cdot \hat{r}$ is applied along the chassis right-vector, where v_{lat} is the lateral velocity component and k_{grip} is a tunable coefficient. This allows controlled drifts at high speed while maintaining stable low-speed handling.

Steering modifies the angular limit of the front wheel. We map the horizontal analog stick input (ranging from -1.0 to 1.0) directly to a steering angle clamp (e.g., ± 30 degrees). Jolt’s constraint solver then automatically resolves the angular velocity, translating the forward torque into an arced trajectory.

Suspension and Slope Adaptation: The raycast suspension serves as the bicycle’s shock absorbers. We configured the suspension’s rest length and spring stiffness so that the rigid body floats slightly above the physical wheel ray hit points. This absorbs the immense vertical forces generated when landing from the jump ramp, preventing the chassis from flipping uncontrollably upon impact. To resolve initial collision bugs where the bicycle physics body clipped into terrain slopes, we implemented a robust pitch-adaptation system. Using raycasts, we actively align the bike’s orientation with the underlying terrain slope, allowing the physics engine to properly resolve collisions for all parts of the chassis and preventing it from becoming embedded in ramps.

Friction was heavily tuned. Out-of-the-box rigid-body friction feels sluggish for a vehicle. We manually implemented a longitudinal vs. lateral friction curve constraint. As longitudinal velocity increases, lateral grip is artificially reduced, allowing the player to initiate controlled slides around tight corners, creating a nimble, arcade-like game feel, taking cues from the responsive handling of *DiRT Rally* [Codemasters 2015] and *Forza Horizon 5* [Playground Games 2021; Tector 2014].

3.5.3 Trigger Mechanisms & Win/Fail Logic. The central gameplay loop revolves around a collection mechanic.

- **Boosters:** Scattered throughout the level are booster entities equipped with Trigger volume colliders. When the bicycle intersects a booster, the `PhysicsSystem` dispatches an overlap event. The event handler subsequently flags the booster for deletion. To prevent fatal table lock crashes

when mutating Flecs entities mid-iteration (e.g., removing `PhysicsBody` components while the system is iterating), we utilized Flecs’ deferred operations (`defer_begin()` and `defer_end()`). This ensures component structural changes are safely queued and executed at the end of the frame.

- **Level Completion & Death Condition:** Once all boosters are collected, the user’s state transitions to “Boost Mode”, temporarily augmenting the bicycle’s maximum torque and decreasing air-resistance logic. This enables the player to ascend the jump ramp at maximum velocity. We have also implemented a robust death condition loop. If the player fails to clear the ramp and crashes off the bike, the game detects the collision failure and triggers the death sequence, seamlessly resetting the level and preventing the player from phasing through boundaries or entering a soft-lock.
- **Win Condition:** A massive, invisible trigger volume is positioned just outside the enclosing walls. Successfully clearing the wall using the ramp causes the bicycle to intersect this volume, firing the “Level Complete” event. This immediately unloads the current Flecs component space and initializes the subsequent scene via the `SceneManager`.

3.5.4 Camera System Architecture. To effectively showcase the high-speed bicycle mechanics, we implemented a dynamic third-person camera that smoothly tracks the player without causing motion sickness. Simply locking the camera’s position and rotation strictly to the Jolt `VehicleConstraint` rigid body results in an erratic, unplayable experience, as the bicycle’s chassis violently pitches and rolls over uneven terrain.

To solve this, our camera operates as an independent Flecs entity that tracks a target position slightly behind and above the player’s `WorldTransform`. We utilize a custom spring-damper mathematical model to interpolate the camera’s current position towards the target position every frame.

For rotation, we completely decouple the camera’s pitch and roll from the bicycle’s physics. We instead calculate a “LookAt” matrix. We extract the bicycle’s current translation vector and project a focal point slightly ahead of the front wheel. The camera then calculates a normalized direction vector pointing from its spring-damped position to this focal point:

The camera position $\mathbf{p}_{\text{cam}}$ is computed in spherical coordinates relative to a target point $\mathbf{t}$ offset above the player, parameterised by orbital distance d , yaw θ , and pitch ϕ . Each parameter is smoothed per frame via an exponential spring-damper with stiffness $s = 6.0$, interpolating towards the target orbit as $\theta(t + \Delta t) = \theta(t) + (\theta_{\text{target}} - \theta(t)) \cdot s \cdot \Delta t$, and analogously for ϕ and d . The final view matrix is constructed via `lookAt( $\mathbf{p}_{\text{cam}}$ ,  $\mathbf{t}$ ,  $\hat{\mathbf{u}}_{\text{dyn}}$ )`. This spring-damped orbit ensures the player remains comfortably centered in the frame, while the physical bicycle is allowed to dynamically bounce and bank independently beneath the focal point.

4 Evaluation

To rigorously assess the technical soundness of our custom engine, we established specific testing methodologies targeting rendering efficiency, OS-level driver overhead, and hardware scaling.

4.1 Testing Environment & Hardware Specifications

Our benchmarking was conducted across two distinct hardware profiles to evaluate both architectural scaling and operating system overhead.

System 1: OMEN15 Laptop (Dual-Boot)

- **CPU:** AMD Ryzen 7 4800H (16 threads)
- **GPU:** NVIDIA GeForce GTX 1660 Ti
- **RAM:** 16.0 GiB (3200MT/s)
- **OS 1 (Linux):** Ubuntu 24.04.4 LTS (Kernel 6.17.0-23-generic)
- **OS 2 (Windows):** Windows 11 Home Build 26200

System 2: UNI Lab PC

- **CPU:** 12th Gen Intel Core i7-12700 (20 threads)
- **GPU:** NVIDIA GeForce RTX 4070
- **RAM:** 30 GiB
- **OS:** Red Hat Enterprise Linux 9.7

4.2 Methodology

We performed a deterministic, automated camera fly-through of our heaviest scene geometry to ensure identical workloads across all tests. We focused on measuring Average FPS under two primary conditions: Frustum Culling Enabled vs Disabled.

- **OS Comparison (OMEN15):** To evaluate the Vulkan API’s driver implementation differences, we ran identical Linux and Windows binaries on the exact same OMEN15 hardware. We utilized MangoHud for Linux telemetry and CapFrameX for Windows telemetry.
- **Hardware Scaling (Linux):** To test the engine’s ability to utilize high-end hardware, we compared the mid-range GTX 1660 Ti against the high-end RTX 4070, executing the same Linux binary.

4.3 Results: Frustum Culling & OS Comparison

The OMEN15 dual-boot testing yielded clear performance data regarding both our engine optimizations and the underlying Vulkan drivers.

Ubuntu 24.04 performed slightly worse than Windows 11 with culling off (111.6 FPS vs 112.0 FPS), but significantly outperformed Windows 11 when culling was enabled (113.5 FPS vs 108.6 FPS). On Linux, enabling Frustum Culling improved the average framerate from 111.6 FPS to 113.5 FPS, confirming that our CPU-side culling successfully reduces Vulkan submission overhead. Conversely, the performance regression on Windows when Frustum Culling was enabled indicates potential differences in how the Windows NVIDIA Vulkan driver handles dynamic command buffer recording and CPU-side branching compared to the Linux driver.

4.4 Results: Hardware Scaling

The hardware scaling test evaluated how the engine performs on high-end desktop components versus mobile architectures.

While the OMEN15 produced 111.6 FPS (Off) and 113.5 FPS (On), the high-end UNI Lab PC (RTX 4070) initially delivered a locked 100.0 FPS in both tests. This initial lock was traced back to the engine’s default Vulkan presentation mode (VK_PRESENT_MODE_FIFO_KHR), which enforced V-Sync and

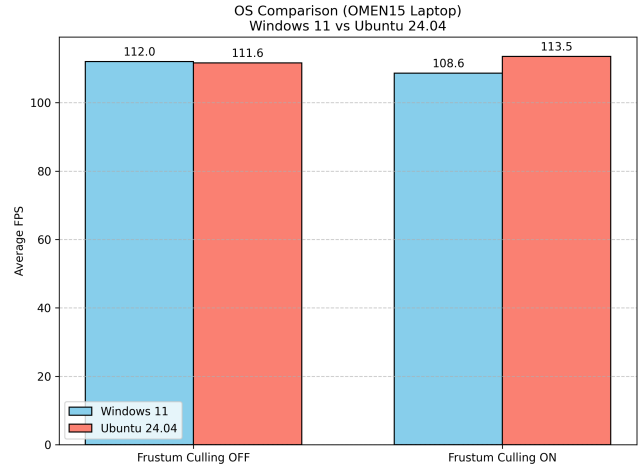


Fig. 7. **OS Comparison.** Average FPS on identical OMEN15 hardware (GTX 1660 Ti) comparing Windows 11 and Ubuntu 24.04.

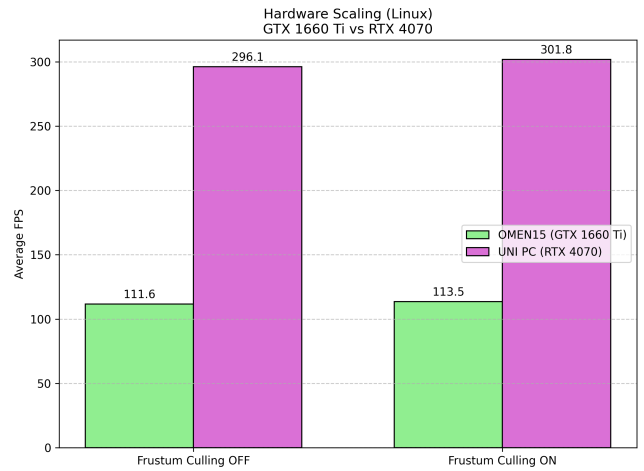


Fig. 8. **Hardware Scaling.** Average FPS on Linux comparing the OMEN15 Laptop (GTX 1660 Ti) and the UNI Lab PC (RTX 4070).

artificially capped the framerate to the lab monitor’s 100Hz refresh rate. To accurately measure the hardware limits, we explicitly modified the RHI layer to request an uncapped presentation mode (VK_PRESENT_MODE_IMMEDIATE_KHR or VK_PRESENT_MODE_MAILBOX_KHR).

With the V-Sync limit removed, the RTX 4070 demonstrated massive processing headroom, skyrocketing to 296.1 FPS (Off) and 301.8 FPS (On). This nearly 2.6x performance delta compared to the GTX 1660 Ti definitively proves that the engine’s underlying architecture scales exceptionally well on high-end desktop hardware when not bottlenecked by synchronization limits.

5 Discussion

Building a custom Vulkan engine from the ground up to support a specialized vehicular game was an immensely challenging but rewarding endeavor. Rather than battling the generalized rigidity of an engine like Unreal, we were able to surgically optimize our collision detection and memory layout for exactly what we needed.

Handling Feedback: One of the key improvements from earlier assignments was the overhaul of the shadow mapping system. We received critical feedback highlighting the lack of shadow hardware PCF (resulting in massive errors when scaling resolution) and the absence of textureProj lookups. We directly addressed this by entirely rewriting the shadow shader bindings, ensuring proper perspective division via textureProj, and binding sampler2DShadow to enable native GPU filtering. The results completely eliminated the aliasing artifacts.

Engine Limitations: Our decision to use pure ECS (Flecs) came with a steep learning curve. While it eliminated OOP-related spaghetti code, debugging state mutations required building custom ImGui visualisations, as standard debugger step-throughs were often unhelpful inside generic system iterators.

Additionally, the initial challenges with gameplay integration for our Particle System have been resolved, allowing for synchronized visual effects (like booster explosions and tire smoke) directly tied to event triggers. Similarly, the successful integration of the Miniaudio AudioSystem significantly elevates the game feel, ensuring that collecting boosters and landing jumps are accompanied by impactful, low-latency auditory feedback.

6 Conclusion

The Steer Engine successfully demonstrates that a targeted, performance-centric architecture can yield rapid, flexible development for specialized games. By marrying the data-locality of Flecs ECS with the raw performance of Vulkan and Jolt Physics, we established a pipeline capable of processing complex level geometry, PBR lighting, and robust vehicular dynamics.

Having successfully integrated full skeletal animation IK and a robust sound system, the demo game delivers a highly polished experience. The core loop proves the engine's capability to seamlessly handle event-driven gameplay logic, concurrent physics simulation, and dynamic audio-visual feedback simultaneously. This loop consists of traversing the level, collecting boosters, gaining speed, and clearing the ramp. The project succeeded in its primary goal: establishing a technical foundation that out-performs generalised architectures for this specific domain.

Acknowledgments

We thank our supervisor Rafael Kuffner dos Anjos, the teaching staff, our classmates, and our families for their invaluable support and guidance.

References

- P. Beckmann and A. Spizzichino. 1963. The Scattering of Electromagnetic Waves from Rough Surfaces. Macmillan.
- CD Projekt Red. 2020. Cyberpunk 2077. CD Projekt.
- Codemasters. 2015. DiRT Rally. Codemasters.
- R. L. Cook and K. E. Torrance. 1982. A Reflectance Model for Computer Graphics. ACM Transactions on Graphics.

- Omar Cornut. 2025. Dear ImGui.
- Erwin Coumans. 2025. Bullet Real-Time Physics Simulation.
- Criterion Games. 2022. Need for Speed Unbound. Electronic Arts.
- Richard Fabian. 2018. Data-Oriented Design.
- Timothy Ford. 2017. Overwatch Gameplay Architecture and Netcode. GDC 2017.
- Marcus Geelnard. 2025. GLFW.
- Axel Gneiting. 2016. The devil is in the details: idTech 6. SIGGRAPH 2016.
- Jason Gregory. 2018. Game Engine Architecture, Third Edition. CRC Press.
- Guerrilla Games. 2022. Horizon Forbidden West. Sony Interactive Entertainment.
- id Software. 2020. DOOM Eternal. Bethesda Softworks.
- Khronos. 2025. Vulkan Documentation.
- Oliver Mackenzie. 2022. Need for Speed Unbound tech review. EuroGamer.
- Sander Mertens. 2025. Flecs: A fast and lightweight C/C++ ECS. <https://github.com/SanderMertens/flecs>
- Nintendo EAD. 2014. Mario Kart 8. Nintendo.
- NVIDIA. 2025. NVIDIA PhysX SDK 4.0.
- Playground Games. 2021. Forza Horizon 5. Xbox Game Studios.
- Jorrit Rouwe. 2021. Physics Animation in Horizon Forbidden West. GDC 2021.
- Jorrit Rouwe. 2025. Jolt Physics. <https://github.com/jrouwe/JoltPhysics>
- Christophe Schlick. 1994. An Inexpensive BRDF Model for Physically-Based Rendering. Computer Graphics Forum.
- Graham Sellers. 2015. OpenGL SuperBible. Addison-Wesley.
- Chris Tector. 2014. Forza Motorsport 5: Next-Generation Physics. GDC 2014.

A Project management

A.1 Supervisor and Theme

Our group was uniquely formed by the teaching staff through the strategic merger of three distinct project pitches: a high-speed bicycle action game, an endless-runner title, and a realistic rocket launch simulator. This synthesis of ideas presented both a creative challenge and a unique technical opportunity. We were assigned Rafael Kuffner dos Anjos as our supervisor, whose expertise in high-performance rendering and real-time systems aligned perfectly with our technical ambitions.

To ensure a cohesive and achievable project scope, we diplomatically consolidated our vision during our initial meetings. We prioritized the high-performance bicycle dynamics as the core gameplay loop, integrated the rocket launch concept as an immersive environmental backdrop, and respectfully pivoted away from the endless-runner mechanics to maintain architectural focus. Throughout the development cycle, we held regular bi-weekly meetings with Rafael. These sessions served as a critical "sanity check" for our progress. Rafael's guidance helped the group prioritize the implementation order and manage project milestones effectively. These meetings ensured that our development trajectory remained focused and that our internal deadlines were scientifically sound and achievable.

A.2 Group organization

The group dynamic was generally very positive, characterized by a shared commitment to technical excellence. We adopted an agile development strategy, utilizing Git and GitHub for version control and Trello for granular task tracking. To maintain momentum, Samuil Georgiev and Sicheng Chen shared managerial responsibilities. Samuil established the high-level roadmap and Gantt charts, while Sicheng managed day-to-day agile sprints.

Communication and Meetings: We established a cohesive technical vocabulary early in the semester and communicated primarily through a dedicated Discord server. This server was partitioned into functional channels (e.g., #vulkan-renderer, #physics-engine, #ecs-logic) to ensure discussions remained focused. We held internal "stand-up" meetings twice weekly: once on Mondays to set the sprint goals and once on Thursdays to prepare for our supervisor session. These internal meetings allowed us to identify "blocking" issues early, such as when the initial Vulkan buffer abstractions were incompatible with the Flecs component storage.

Work Distribution: Labor was divided based on individual technical interests. Sicheng and Samuil focused on the low-level rendering and physics cores, while Qidan, Junxian, and Chenghao managed the engine architecture, UI/tools, and skeletal animation systems respectively. We adhered to a strict Git branching strategy: all features were developed on feature/ branches and required a peer review before merging into main. This prevented regressions and ensured that the master build remained stable.

Conflict Resolution: Architectural disagreements were inevitable, particularly regarding the trade-off between ECS flexibility and raw physics performance. We resolved these by referring back to our core requirements: if a proposed abstraction introduced unnecessary CPU overhead in the vehicle simulation,

it was rejected in favor of a more data-oriented approach. This rigorous adherence to performance-driven design helped us maintain a unified technical vision even during high-pressure milestones.

A.3 Planning and execution

Our project's trajectory, visualised through the Gantt charts in Figure 9, underscores the evolution from a structured initial roadmap to a responsive, agile workflow. This comparison highlights our dynamic task reallocation, particularly during the transition from the "Silo" prototype to the final Bicycle Action Game.

Initial Plan and MVP: At the beginning of the semester, we allocated specific milestones for engine architecture and physics integration. However, we underestimated the complexity of unifying these systems under a single ECS paradigm. Creating the pipeline that translated raw level data into Flecs entities and Jolt physics bodies took nearly four weeks instead of the planned two. Because we front-loaded these architectural foundations, we were able to absorb the delay without sacrificing the "Minimum Viable Product" (MVP) deadline. By Week 9, we had a steerable chassis and a basic rendering pass, which allowed us to begin gameplay iteration.

The Mid-Semester Pivot: The most significant test of our planning occurred mid-semester. Our original concept was an atmosphere-heavy game set in an underground "Silo." It became clear that an enclosed environment fought the strengths of our engine, which excelled at high-speed vehicular dynamics and expansive lighting. We made the decisive choice to scrap the underground level and transition to the current Bicycle demo. Because our architecture was heavily decoupled via Flecs, we were able to reuse our rendering and physics cores entirely. This agile response saved the project from a "technical mismatch" and allowed us to focus on polishing the bicycle's handling and ramps.

Final Push and Challenges: As the project neared completion, we faced challenges related to inconsistent attendance during the Easter break. We mitigated this by transitioning to entirely online meetings and maintaining a rigorous "backlog" on Trello. This ensured that even when members were away, they had clear technical specifications to follow. The final two weeks were dedicated to "Game Feel" polish: tuning the Jolt suspension, refining the shadow hardware PCF, and implementing the skeletal IK for the rider.

B Self-reflection and soft skills acquired.

The development of Steer Engine was a transformative experience that extended far beyond technical implementation. From a project management perspective, the greatest lesson was the value of architectural foresight. By investing heavily in a researched ECS architecture (Flecs) during the first two weeks, the group avoided the catastrophic refactors that often plague student projects when merging complex physics and rendering systems. The team learned to navigate "high-pressure peaks," such as the transition from the original silo concept to the bicycle demo, by maintaining open, honest communication and prioritizing a Minimum Viable Product (MVP) that showcased the engine's unique strengths. These skills, including agile pivoting, cross-disciplinary technical communication, and

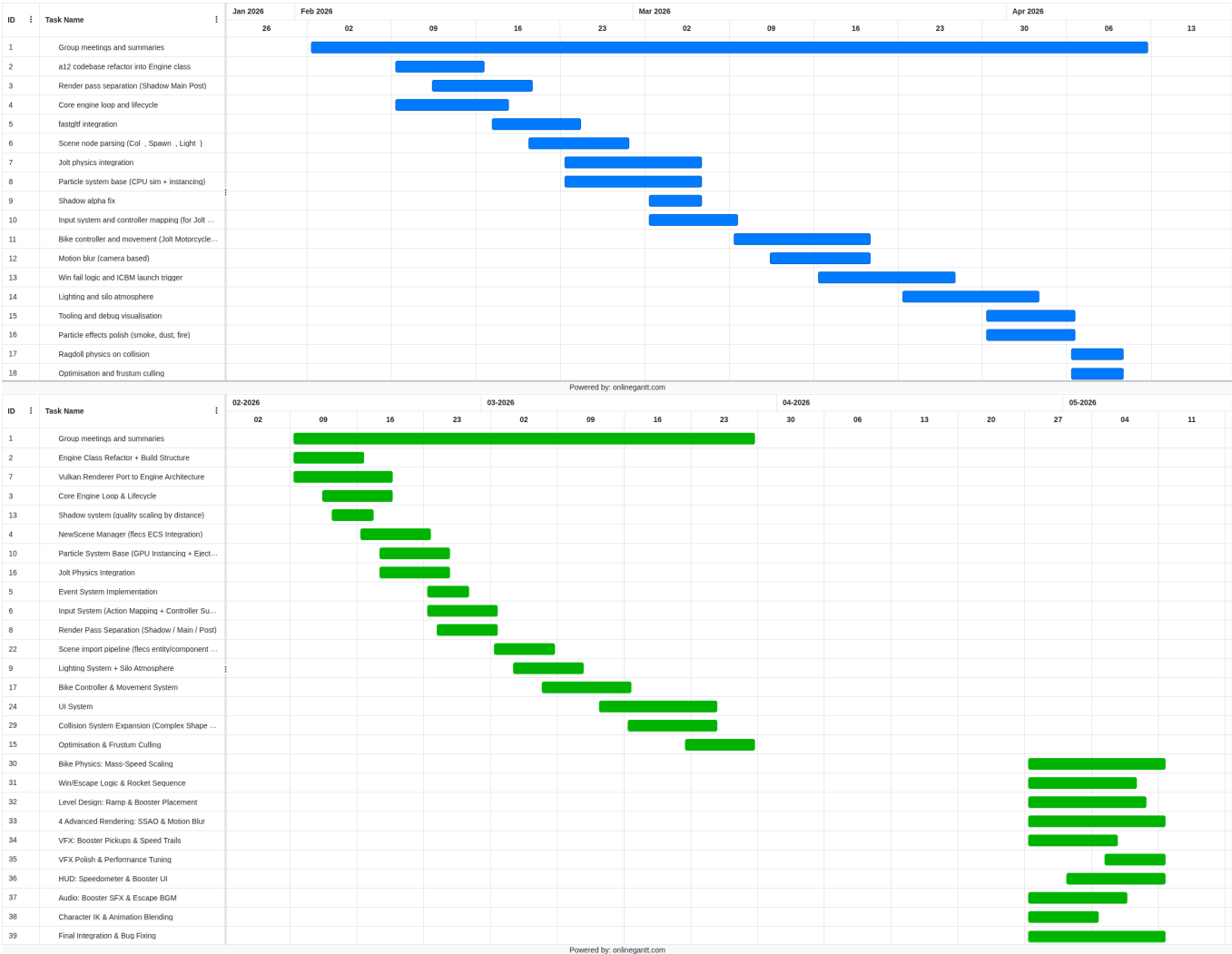


Fig. 9. **Project Planning and Timeline Comparison.** Top: Initial planning Gantt chart. Bottom: Actual execution Gantt chart. The comparison between the initial plan and final execution highlights our dynamic task reallocation and agile development process throughout the engine's lifecycle.

rigorous version control, are directly applicable to the professional software industry.

Sicheng Chen (Renderer & Physics Logic): Managing the entity system and the scene manager through Flecs integration taught the value of data-oriented design. Sicheng focused on the advanced rendering pipeline, implementing Cascaded Shadow Maps, Screen-Space Reflections (SSR), and Image-Based Lighting (IBL). Mastering the Vulkan pipeline presented a steep learning curve, especially when resolving complex aliasing issues. Beyond the core renderer, developing the post-processing suite (Bloom, Motion Blur, Chromatic Aberration) and the advanced torque-curve-based bike controller provided a deep understanding of "game feel." The mid-semester pivot taught that being emotionally detached from an initial design is necessary if the technical data suggests a better

path. This member learned to balance visual fidelity with simulation performance while acquiring significant soft skills in technical documentation.

Samuil Georgiev (Management & System Integration): The primary takeaway from this project was the critical importance of maintaining a consistent architectural vision. Managing the integration of Jolt Physics and the initial Vulkan renderer skeleton required constant mediation between different subsystems to ensure data-locality. Samuil learned to balance managerial duties, such as Gantt chart tracking and the attempted Mac OS port, with hands-on debugging of low-level Linux driver issues. The experience of leading the production of the engine showcase video and technical poster demonstrated the value of high-impact communication. This member found that proactive encouragement and clear API contracts

are more effective for maintaining momentum than reactionary firefighting.

Qidan Guo (Engine Architecture & Physics Integration): Building the core engine architecture and the build system provided deep insights into the foundations of high-performance software. Implementing the model loader and the camera system required a rigorous understanding of coordinate spaces and character-vehicle binding. Qidan learned that engine stability is heavily dependent on frame rate management and deterministic tick logic. Developing the Inverse Kinematics (IK) solver and the basic bike controller provided practical experience in bridging the gap between mathematical models and interactive gameplay logic.

Junxian Li (UI & Editor Tooling): Developing the Visual UI Editor and the runtime UI authoring pipeline demonstrated the importance of developer-facing tools. It was learned that a production-ready engine requires an intuitive tooling layer, such as the real-time scene editing system and ImGui-based manipulation tools. Junxian acquired significant experience in GPU-based interaction, implementing ray-cast and bounding-box editor selection. Beyond the UI, developing the particle system, audio system, and frustum culling optimizations reinforced the need for efficient resource management in a real-time environment.

Chenghao Zhu (Skeletal Animation System): Integrating the ozz-animation library for the runtime animation playback system presented a significant technical challenge. Chenghao learned the complexities of animation data loading, keyframe sampling, and bone hierarchy evaluation. Implementing the skinned mesh deformation system and the renderer bone matrix upload pipeline provided deep insights into the vertex shader pipeline. This member developed strong skills in integrating third-party middleware into a custom ECS framework, ensuring that animated characters were synchronized with the underlying physics simulation.

Summary of Technical Contributions

Sicheng Chen - Flecs Entity System Integration, Scene Manager Architecture, Light Source and Emissive Object Creation, Cascaded Shadow Maps (CSM), Third-person Camera Barrier Transparency, Skybox and Image-Based Lighting (IBL), Screen-Space Reflection (SSR), Screen-Space Ambient Occlusion (SSAO), Normal Map Channel Support, Post-processing Suite (Bloom, Motion Blur, Chromatic Aberration, Lens Distortion), Torque-Curve-based Bike Controller, Steer-lean blending, FOV auto-alignment systems.

Samuil Georgiev - Project Management (Gantt charts & initial distribution), Linux Support & Cross-Platform Compatibility, Mac OS port attempt, Jolt Physics integration (physics & collisions), Input System & Controller support, Initial Renderer skeleton, Original Shadow implementation, Engine Showcase Video Editing & Voiceover, Poster, Report Writing, Benchmarking Suite and Performance Analysis, Git Version Control Management.

Qidan Guo - Engine Architecture, Model Loader, Build System, Camera System (Character Binding), Animation System, Inverse Kinematics (IK) System, Character Import & Rendering Pipeline, Character-Vehicle Binding, Bike Controller (basic speed and force control, stop and go logic), Engine Tick & Frame Rate Management.

Junxian Li - Visual UI Editor system, Runtime UI system and UI authoring pipeline, Engine UI tooling layer, UI animation system, Runtime debugging interface, Content Browser, Real-time scene editing system, Scene save/load serialization workflow, ImGui-based transform manipulation tools, Particle system, Trigger system, Ray-cast and Bounding-box editor interaction, Debug rendering tools, Frustum culling optimisation, Audio System.

Chenghao Zhu - Runtime Animation Playback System (ozz-animation), Animation Data Loading and Registration Pipeline, Keyframe Sampling and Interpolation System, Bone Hierarchy Evaluation System, Skinned Mesh Deformation System, Renderer Bone Matrix Upload Pipeline, Animated Character Runtime Integration.

C How ethical issues are addressed

No specific ethical issues are involved in the scope of this generic game engine project. .